

HOTEL ACCOMMODATION AND RESERVATION MANAGEMENT USING C++

Assignment Title: Hotel Accommodation and Reservation Management Using C++

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Problem Statement and Problem Formulation	1
2	Objectives and Expected Outcomes	2
3	Requirements, Constraints and Assumptions	3
4	Application of Relevant Course Knowledge	4
5	Proposed Solution and Methodology	5
6	System Design	6
7	Algorithm and Pseudocode	7
8	Flowchart	8
9	Implementation	9
10	Environment and Tools Used	10
11	Debugging Approach	11
12	Test Cases and Results	12
13	Execution Screenshots and Outputs	13
14	Results and Validation	14
15	Analysis, Comparison and Justification	15
16	Broader Considerations / SDG Relevance	16
17	Individual Contribution	17
18	Individual Reflection	18
19	Conclusion	19
20	Limitations and Future Improvements	20
21	References	21

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

Hotel reservation management involves handling guests, rooms, reservations, hotel services and billing. Managing these operations manually can result in incorrect room allocation, duplicate reservations, billing errors difficulty in maintaining guest information.

The proposed Hotel Accommodation and Reservation Management System is a menu-driven C++ application developed using Object-Oriented Programming principles. The system enables hotel staff to register guests, display available rooms, make and cancel reservations, add hotel services, calculate bills and generate reservation details.

The application uses different room and service categories through inheritance. Abstract classes, virtual functions, pointers and runtime polymorphism are incorporated to make the system modular, reusable extensible.

1.2 Problem Formulation

- Register and manage guest information.
- Maintain different categories of hotel rooms.
- Display room availability.
- Create and cancel reservations.
- Add additional hotel services.
- Calculate the total customer bill.
- Generate reservation details.
- Prevent reservation of already occupied rooms.
- Demonstrate inheritance and runtime polymorphism.
- Detect and correct programming errors during debugging.

2. OBJECTIVES AND EXPECTED OUTCOMES

2.1 Objectives

- To develop a menu-driven hotel reservation application using C++.
- To apply object-oriented programming concepts in a practical application.
- To implement different room categories using inheritance.
- To use abstract classes and pure virtual functions.
- To demonstrate multiple and hierarchical inheritance where appropriate.
- To use virtual base classes to resolve duplicate inheritance.
- To implement runtime polymorphism using base-class pointers.
- To manage guest reservations efficiently.
- To calculate room and service charges accurately.
- To identify and correct common programming errors through debugging.

2.2 Expected Outcomes

- Successfully register guests.
- Display available rooms.
- Create valid reservations.
- Reject reservations for occupied rooms.
- Cancel existing reservations.

- Add additional services.
- Calculate the final bill.
- Display reservation information.
- Demonstrate dynamic polymorphism.
- Handle invalid inputs and runtime errors appropriately.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

- Guest registration
- Room listing
- Room availability checking
- Room reservation
- Reservation cancellation
- Hotel service selection
- Bill calculation
- Reservation details
- Menu-based navigation

3.2 Non-Functional Requirements

- Easy to use
- Modular
- Maintainable
- Reusable
- Efficient
- Error-resistant
- Extensible

3.3 Constraints

- The application is developed using C++.
- Data is maintained during program execution.
- The application uses console-based interaction.
- The system does not require an external database.
- Only registered guests can make reservations.

3.4 Assumptions

- Each reservation is associated with one guest.
- A room cannot be reserved by multiple guests simultaneously.
- Room charges are calculated according to room category.
- Additional services are charged separately.
- Cancellation releases the room for future reservations.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE

C++ Concept	Application
Classes and Objects	Guest, Room, Reservation and Service
Encapsulation	Data members and member functions

Inheritance	Different room and service categories
Abstract Class	Base Room and Service classes
Pure Virtual Function	Room and service charge calculation
Virtual Function	Runtime polymorphism
Pointers	Dynamic object management
Runtime Polymorphism	Different room types through base pointer
Virtual Base Class	Avoiding duplicate base-class instances
Function Overriding	Room-specific charge calculation
Constructors	Object initialization
Destructors	Resource cleanup
Exception Handling	Invalid operations/input handling

5. PROPOSED SOLUTION AND METHODOLOGY

The proposed system follows a modular object-oriented approach. The application is divided into Guest Management, Room Management, Reservation Management, Hotel Services, Billing, and Debugging and Validation modules.

5.1 Guest Management

Responsible for guest registration, guest identification and guest information management.

5.2 Room Management

Responsible for maintaining room details, displaying room categories, checking availability and calculating room charges.

5.3 Reservation Management

Responsible for creating and cancelling reservations, assigning rooms to guests and maintaining reservation information.

5.4 Hotel Services

Responsible for food, laundry, spa and other additional services.

5.5 Billing

Responsible for calculating room charges, service charges and generating the final bill.

5.6 Debugging and Validation

Responsible for detecting invalid input, preventing duplicate reservations, handling invalid room numbers and validating reservation operations.

6. SYSTEM DESIGN

6.1 Class Design

The system consists of Guest, Staff, Reservation, Hotel, Room and Service classes. Room and Service are designed as abstract bases, with specialized derived classes for different categories.

Class hierarchy:

Person → Guest / Staff → Reservation → Hotel → Room (Abstract) → StandardRoom /

DeluxeRoom / SuiteRoom

Service (Abstract) → FoodService / SpaService

6.2 Runtime Polymorphism

A base-class pointer can refer to different derived room objects. When a virtual function is called through the pointer, the appropriate derived-class implementation is selected at runtime.

6.3 Virtual Base Class

Virtual inheritance can be used where two inheritance paths lead to the same base class. This prevents multiple copies of the common base-class portion from being created.

7. ALGORITHM AND PSEUDOCODE

7.1 Main Algorithm

Step: Start the application.

Step: Display the main menu.

Step: Accept the user's choice.

Step: If Guest Registration is selected, collect guest details.

Step: If Room Availability is selected, display available rooms.

Step: If Reservation is selected, verify guest and room availability.

Step: Create the reservation if the room is available.

Step: If Services is selected, display available services.

Step: Add selected services to the reservation.

Step: Calculate room and service charges.

Step: If cancellation is selected, cancel the reservation and release the room.

Step: Display reservation details when requested.

Step: Continue until the Exit option is selected.

Step: Terminate the program.

7.2 Pseudocode

START

Initialize hotel system

REPEAT

 Display menu

 Read choice

 IF choice = 1

 Register guest

 ELSE IF choice = 2

 Display available rooms

 ELSE IF choice = 3

 Read guest ID

 Read room number

 IF room is available

 Create reservation

 Mark room as occupied

 ELSE

 Display "Room not available"

```

    END IF
ELSE IF choice = 4
    Read reservation ID
    Cancel reservation
    Release room
ELSE IF choice = 5
    Display hotel services
    Add selected service
ELSE IF choice = 6
    Calculate room charge
    Calculate service charge
    Display total bill
ELSE IF choice = 7
    Display reservation details
ELSE IF choice = 8
    EXIT
ELSE
    Display "Invalid choice"
END IF
UNTIL choice = 8

```

STOP

8. FLOWCHART

START → Initialize System → Display Menu → Read User Choice → Guest Registration / Room Availability / Reservation → Service Selection → Calculate Bill → Display Reservation → Exit Selected? → STOP

9. IMPLEMENTATION

The implementation is carried out using C++ classes and object-oriented principles. The major implementation components include abstract base classes, derived room classes, virtual functions, pointers and dynamic object management.

9.1 Base Abstract Class

```

class Room {
public:
    virtual double calculateCharge(int days) = 0;
    virtual string getRoomType() = 0;
    virtual ~Room() {}
};

```

9.2 Derived Room Class

```

class DeluxeRoom : public Room {
public:
    double calculateCharge(int days) override {
        return days * 3000;
    }

    string getRoomType() override {

```

```

        return "Deluxe Room";
    }
};

```

9.3 Runtime Polymorphism

```
Room *r = new DeluxeRoom();
```

```

cout << r->getRoomType() << endl;
cout << r->calculateCharge(3) << endl;

```

```
delete r;
```

The complete source code should be included in the final GitHub submission.

10. ENVIRONMENT AND TOOLS USED

Component	Technology
Programming Language	C++
Programming Paradigm	Object-Oriented Programming
IDE	Visual Studio Code / Code::Blocks
Compiler	GCC / MinGW
Version Control	Git
Repository	GitHub
Documentation	Microsoft Word
Operating System	Windows

11. DEBUGGING APPROACH

Debugging was performed to identify syntax, logical and runtime errors.

Error	Cause	Solution
Compilation error	Incorrect class declaration	Correct class syntax
Ambiguous inheritance	Multiple copies of base class	Use virtual inheritance
Incorrect output	Wrong calculation	Correct billing logic
Room double booking	Availability not checked	Validate room status
Runtime crash	Invalid pointer operation	Correct pointer management
Polymorphism failure	Missing virtual function	Declare function as virtual
Memory leak	Dynamically allocated object not deleted	Use delete
Incorrect overriding	Function signature mismatch	Use override

12. TEST CASES AND RESULTS

Test Case	Input	Expected Result	Actual Result	Status
TC01	Register valid guest	Guest registered	Guest registered	Pass
TC02	Select available room	Room can be reserved	Room reserved	Pass
TC03	Select occupied room	Reservation rejected	Reservation rejected	Pass
TC04	Cancel reservation	Room becomes available	Room released	Pass
TC05	Add food service	Service charge added	Charge added	Pass
TC06	Generate bill	Correct total displayed	Correct total displayed	Pass

TC07	Invalid menu choice	Error message displayed	Error message displayed	Pass
TC08	Invalid room number	Invalid room message	Message displayed	Pass

13. EXECUTION SCREENSHOTS AND OUTPUTS

Screenshot 1 – Main Menu

```

Hotel Reservation Management System                                01_main_menu

===== HOTEL RESERVATION MANAGEMENT SYSTEM =====
1. Register Guest
2. Display Guests
3. Display Room Availability
4. Make Reservation
5. Add Hotel Service
6. View Reservation / Bill
7. Cancel Reservation
8. Hotel Utilization Report
9. Exit
Enter choice:

```

Screenshot 2 – Guest Registration and Guest Details

```

Hotel Reservation Management System                                02_guest_registratio

Enter choice: 1
Enter Guest ID: 101
Enter Name: Arun Kumar
Enter Phone: 9876543210
Enter Email: arun@gmail.com
Guest registered successfully.

Enter choice: 2
Guest - ID: 101 | Name: Arun Kumar | Phone: 9876543210 | Email: arun@gmail.com

```

Screenshot 3 – Room Availability

```
Hotel Reservation Management System                                03_room_availability

Enter choice: 3

Room No | Type | Rate | Status
101 | Standard | Rs.2500.00 | Available
102 | Standard | Rs.2500.00 | Available
201 | Deluxe | Rs.4000.00 | Available
202 | Deluxe | Rs.4000.00 | Available
301 | Suite | Rs.6500.00 | Available
```

Screenshot 4 – Successful Reservation

```
Hotel Reservation Management System                                04_successful_reserv

Enter choice: 4
Guest - ID: 101 | Name: Arun Kumar | Phone: 9876543210 | Email: arun@gmail.com
Enter Guest ID: 101
Enter Room Number: 201
Enter number of days: 2
Reservation created. ID: 1001
```

Screenshot 5 – Occupied Room Validation

```
Hotel Reservation Management System                                05_occupied_room_val

Enter choice: 4
Enter Guest ID: 101
Enter Room Number: 201
Invalid input. Try again.
Room unavailable or invalid.

Validation: the already reserved room cannot be booked again.
```

Screenshot 6 – Hotel Service Selection

```
Hotel Reservation Management System                                06_hotel_service

Enter choice: 5
Enter Reservation ID: 1001
1. Food  2. Laundry  3. Spa
Choose service: 1
Service added.

Food Service charge: Rs.500.00
```

Screenshot 7 – Bill Generation

```
Hotel Reservation Management System                                07_bill_generation

Enter choice: 6
Enter Reservation ID: 1001
Reservation ID: 1001
Guest: Arun Kumar (101)
Room: 201 - Deluxe
Days: 2
Status: Active
Room Cost: Rs.8000.00
Services: Rs.500.00
Total Bill: Rs.8500.00
```

Screenshot 8 – Reservation Cancellation

```
Hotel Reservation Management System                                08_cancellation

Enter choice: 7
Enter Reservation ID: 1001
Reservation cancelled.

Enter choice: 3
201 | Deluxe | Rs.4000.00 | Available

The cancelled room is available again.
```

The eight screenshots in this section have been changed to match the current program's menu, guest, room, reservation, service, billing and cancellation outputs. Before final submission, replace any sample data with screenshots taken from the exact GitHub version used by the team if your faculty requires direct screen captures.

14. RESULTS AND VALIDATION

The developed system successfully demonstrates hotel accommodation and reservation operations through a menu-driven C++ application. The testing process verified guest registration, room availability, reservation creation, duplicate booking prevention, cancellation, service addition, bill calculation, runtime polymorphism and invalid-input handling.

The test results demonstrate that the proposed system satisfies the major functional requirements.

15. ANALYSIS, COMPARISON AND JUSTIFICATION

A simple procedural implementation could manage hotel reservations using functions and arrays. However, such an approach becomes difficult to maintain when new room types or services are added. The object-oriented approach provides better modularity, reuse, abstraction and extensibility.

Feature	Procedural Approach	Proposed OOP Approach
Code Reusability	Low	High
Maintainability	Moderate	High
Extensibility	Low	High
Inheritance	Not available	Available
Polymorphism	Not directly supported	Supported
Data Encapsulation	Limited	Strong
New Room Types	Requires major changes	New derived class
Debugging	More difficult	Modular debugging

Therefore, the OOP approach is more suitable for the proposed hotel management application.

16. BROADER CONSIDERATIONS / SDG RELEVANCE

The project has reasonable connections to SDG 7 – Affordable and Clean Energy, SDG 11 – Sustainable Cities and Communities, and SDG 12 – Responsible Consumption and Production. The current program mainly manages rooms, guests, reservations and services. In a future version, hotel resource and energy usage could be recorded to support better operational decisions. The system also reduces manual record keeping and can help organize accommodation and service use more systematically.

- Digital transformation
- Efficient resource management
- Reduction of manual errors
- Technology-driven service management
- Scalable software design

17. INDIVIDUAL CONTRIBUTION

Member	Contribution
Member 1	Designed class hierarchy, room management and inheritance implementation.
Member 2	Developed guest registration and reservation management.
Member 3	Developed service management, billing, debugging and testing.

All members participated in integration, debugging, documentation and final validation.

18. CONCLUSION

The Hotel Accommodation and Reservation Management System successfully demonstrates the implementation of a real-world application using C++ Object-Oriented Programming concepts. The system provides facilities for guest registration, room availability, reservation management, cancellation, hotel services and bill calculation.

The use of abstract classes, inheritance, virtual base classes, pointers and runtime polymorphism improves the modularity and extensibility of the application. Debugging and validation helped identify and correct syntax, logical and runtime errors.

The project demonstrates that an object-oriented approach is suitable for developing maintainable and reusable hotel management software.

19. LIMITATIONS AND FUTURE IMPROVEMENTS

19.1 Limitations

- Data is not permanently stored.
- The application uses a console interface.
- No online payment facility is included.
- Multiple users cannot access the system simultaneously.
- No external database is connected.

19.2 Future Improvements

- Integrating MySQL or another database.
- Developing a graphical user interface.
- Adding online payment functionality.
- Implementing user authentication.
- Adding email/SMS reservation notifications.
- Developing an administrator dashboard.
- Adding customer feedback and rating features.
- Creating a web or mobile version.

20. REFERENCES

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley.
2. Robert Lafore, Object-Oriented Programming in C++, 4th Edition, Sams Publishing.
3. Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, C++ Primer, 5th Edition, Addison-Wesley.
4. Paul Deitel and Harvey Deitel, C++ How to Program, Pearson.
5. ISO/IEC, Programming Languages – C++ Standard.
6. GitHub Documentation, Git and repository management resources.